

UNIVERSITY OF EUROPE FOR APPLIED SCIENCES — DUBAI



Faculty of Data Science

PROJECT REPORT

Data Analytics — DA01

Predicting Late Deliveries in Brazilian E-Commerce: A Comparative Machine Learning Study with Class Imbalance Handling Using the Olist Dataset

Prepared by:

Renato Silva • Sandra Raj Pattuvakkaran • Sudarsh Mekkampurath Sajeev

Student IDs: 27732411 , 39755112 , 78109630

Supervisor: Prof. Dr. Nor Azizah Hitam

Submitted: July 2026

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
1: INTRODUCTION.....	3
1.1 Project Title.....	3
1.2 Problem Statement.....	3
1.3 Project Objectives.....	3
1.4 Dataset / Data Source.....	3
1.5 Expected Outcome.....	4
2 LITERATURE REVIEW.....	4
3 METHODOLOGY.....	4
3.1 Dataset Understanding and Preparation.....	4
3.1.1 Dataset Description.....	4
3.1.2 Main Features / Columns.....	5
3.1.3 Data Issues Identified.....	5
3.1.4 Summary of Problems Highlighted.....	5
3.2 Data Cleaning and Initial Analysis.....	5
3.2.1 Data Cleaning Steps Completed.....	5
3.2.2 How Issues from Checkpoint 2 Were Solved.....	6
3.2.3 Initial Analysis and Visualizations.....	6
3.2.4 Initial Findings from the Data.....	6
4: ANALYSIS AND MACHINE LEARNING.....	7
4.1 Final Dataset and Selected Features.....	7
4.2 Target Variable.....	7
4.3 Machine Learning Model Selected.....	8
4.4 Model Training and Testing Process.....	8
4.5 Model Results.....	8
4.6 Explanation of Model Results.....	9
FINAL FINDINGS AND CONCLUSION.....	10
5.1 Final Findings.....	10
5.2 Recommendations.....	10
5.3 Project Limitations.....	10
5.4 Future Improvements.....	11
5.5 Conclusion.....	11
REFERENCES.....	11

1: INTRODUCTION

1.1 Project Title

Predicting Late Deliveries in Brazilian E-Commerce: A Comparative Machine Learning Study with Class Imbalance Handling Using the Olist Dataset

1.2 Problem Statement

Approximately 8.1% of orders placed on the Olist Brazilian e-commerce platform are delivered after their expected delivery date. In a marketplace of approximately 100,000 orders, this is almost 8,000 poor delivery experiences for customers which resulted in negative reviews and loss of faith in both for separate sellers and the platform as a whole. The central analytical challenge is not only to predict lateness, but to do so in the presence of a substantial class imbalance: Only 8.1% of orders are late, so a naive model that always predicts “on-time” gets it right 91.9% of the time and no late deliveries. There’s no way to flag orders on the platform as at-risk yet before they’re late, meaning logistics teams can only respond to delays in retrospect rather than proactive intervention. This project aims to fill this gap by developing and carefully comparing several models based solely on information available at the time the order is placed.

1.3 Project Objectives

- **Exploratory Data Analysis** — analyze class distribution, delivery-time histograms, correlation heatmap, outlier detection and late-delivery rates per state in Brazil.
- **Feature Engineering** - derive new predictive features from raw order data `order_hour`, `order_day_of_week`, `same_state`, `product_volume` and `payment_ratio`.
- **Class Imbalance Handling** — use SMOTE (Synthetic Minority Oversampling Technique) to training data only, never to the test set, to address the 92:8 on-time/late split.
- **Build and evaluate five ML models** – Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbours, XGBoost, evaluated using F1-score and ROC-AUC instead of precision
- **Statistical Validation** – Paired t-test on 10-fold cross-validation F1 scores to determine which performance differences between models are statistically significant ($p < 0.05$).

1.4 Dataset / Data Source

Field	Details
Dataset Name	Olist Brazilian E-Commerce Public Dataset (enriched as orders_enriched via the companion Data Engineering pipeline)
Source / URL	Kaggle — kaggle.com/datasets/olistbr/brazilian-ecommerce
Size	96,476 delivered orders × 31 engineered columns (from 9 raw relational tables, ~100,000 original orders)
Format	CSV, ingested into PostgreSQL (3NF schema) via an Apache Airflow ETL pipeline, then exported for analysis
License	Public dataset (Kaggle), released by Olist for research and educational use

Table 1.1 — Dataset summary.

1.5 Expected Outcome

The project is expected to deliver: (1) a binary classifier that can flag at-risk orders at the point of purchase; (2) a thorough comparison of five machine-learning approaches, validated with 10-fold cross-validation and paired statistical testing; and (3) a clear picture of which features – especially geography and logistics variables – mostly impact late delivery; and (4) actionable recommendations for the platform’s logistics and seller-operations teams, based on model results instead of general presumptions.

2 LITERATURE REVIEW

Predicting delivery delays is a hot research topic in applied machine learning for e-commerce and logistics supply chain. K  p, K  p and Ko  ak (2025) compared Logistic Regression, XGBoost, CatBoost, and Random Forest to predict delivery delays at different stages of the shipping process, finding that For real-world courier data, boosting-based ensembles outperformed simpler baselines in general. Similarly, Lesmana et al. (2025) applied XGBoost on e-commerce delivery-delay prediction and reported that model performance was very sensitive to the handling of class imbalance, because on-time deliveries dominate most real-world datasets. In particular, they recommended SMOTE and F1/recall-based accuracy but on evaluation, a finding that is directly in line with the approach used in this project. The methodological basis for class-imbalance handling is well established: SMOTE, introduced by Chawla et al. (2002) produces synthetic minority-class samples by interpolating between existing minority

observations, and is the most popular over-sampling technique in imbalanced classification tasks. In terms of modeling, XGBoost (Chen & Guestrin, 2016) is one of the most consistently good performers on structured/tabular data of the kind produced by this project's feature engineering pipeline, which prompted its inclusion as the expected top performer among the five models here compared. Previous supply-chain research also points out that order-level attributes alone often have a limited predictive ceiling. Studies that compare five or more ML algorithms for supplier delivery delay prediction have found that random forest and gradient boosting methods were found to perform better than logistic regression and k-nearest neighbours, but ultimately constrained by the information in the input data for prediction quality— often mentioned external factors such as carrier events, routing and regional infrastructure are missing but important variables. This observation directly informs the interpretation of this project's results in Section 4.6 and the limitations in Section 5.3.

3 METHODOLOGY

3.1 Dataset Understanding and Preparation

3.1.1 Dataset Description

The Olist Brazilian E-Commerce Public Dataset is made of approximately 100,000 real orders placed in the Olist marketplace from 2016 to 2018, organized in nine relational CSV tables (orders, order_items, customers, sellers, products, payments, reviews, geolocation, category_translation) As. In the companion Data Engineering project, tables were ingested into a PostgreSQL database based on a third normal form (3NF) schema, orchestrated by an Apache Airflow DAG that executed extraction, cleaning, validation and loading on a scheduled basis. The output of that pipeline — an orders_enriched table with 96,476 delivered orders and 31 columns including derived KPIs such as delivery_days, is_late, days_late, and revenue_per_order. The direct input to this Data Analytics project and no need for repeated raw ingestion.

3.1.2 Main Features / Columns

Category	Example Columns
Target	is_late (1 = delivered after the estimated date, 0 = on time)
Delivery	delivery_days, days_late, order_estimated_delivery_date
Product	product_weight_g, product_length_cm, product_height_cm, product_width_cm
Order / Payment	freight_value, price, item_count, payment_installments, payment_type
Geography	seller_state, customer_state

Table 3.1 — Main feature categories in orders_enriched.

3.1.3 Data Issues Identified

- Missing delivery dates – 2,965 orders did not have an order_delivered_customer_date (undelivered, cancelled, or in transit), so is_late cannot be calculated for these rows.
- Missing product data – 610 rows of missing product category names and 2 rows of missing dimensions of product.
- Missing review text – 87,656 missing review titles and 58,247 missing review messages (most customers just leave a numeric score without any written feedback).
- Duplicate review records – duplicate review_id values traced to a known bug in Olist's own submission-review system.
- Referential integrity gaps Some products referenced category names (e.g. pc_gamer) that did not exist in category_translation table.
- Class imbalance very large - only ~8% late delivered orders, ~92% on time.

3.1.4 Summary of Problems Highlighted

The three issues described above posed the greatest risk to downstream analysis. First, the undelivered delivery dates would have had to have been resolved before any target variable could be computed – including them would have used misleading or undefined labels. Second, the holes in referential integrity threatened to break the foreign-key constraints of the 3NF PostgreSQL schema built in the Data Engineering companion project, and were programmatically resolved before loading. Third, and most important for this Data Analytics project specifically, the class imbalance (92:8) made standard accuracy a misleading evaluation metric all along – this is the one issue that most influenced the modeling strategy that is followed in Section 4, motivating the choice of SMOTE, F1-score and ROC-AUC over accuracy.

3.2 Data Cleaning and Initial Analysis

3.2.1 Data Cleaning Steps Completed

Step	Action Taken
Missing Values	Filtered to status = 'delivered' with a non-null delivery timestamp (99,441 → 96,476 rows); missing product dimensions imputed with column median.
Duplicates	drop_duplicates(keep='first') applied to review records before aggregating review scores per order.
Outliers	Extreme delivery durations (>200 days) retained as genuine late-delivery cases rather than removed, confirmed via box-plot inspection.

Data Types	All 5 timestamp columns parsed with <code>pd.to_datetime(errors='coerce')</code> .
Feature Engineering	Created <code>delivery_days</code> , <code>days_late</code> , <code>is_late</code> , <code>order_hour</code> , <code>order_day_of_week</code> , <code>same_state</code> , <code>product_volume</code> , <code>payment_ratio</code> , and <code>review_sentiment_label</code> .

Table 3.2 — Data cleaning steps.

3.2.2 How Issues from Checkpoint 2 Were Solved

Checkpoint 2 Issue	Resolution Applied
Missing delivery dates	Filtered to delivered orders with a confirmed delivery timestamp only.
Missing product dimensions/category	Median-imputed dimensions; 'unknown' category label used; missing categories added to the translation table.
Duplicate review IDs	Kept the first occurrence of each <code>review_id</code> before aggregating.
Referential integrity gaps	Missing category names detected and added programmatically prior to database load.
Severe class imbalance	Deferred to the modelling stage: SMOTE applied to training folds only; F1-score and ROC-AUC used as primary metrics instead of accuracy.

Table 3.3 — Mapping of Checkpoint 2 issues to their Checkpoint 3 resolutions.

3.2.3 Initial Analysis and Visualizations

The exploratory analysis included: (1) a class-distribution bar/pie chart, showing the 91.9%/8.1% on-time/late split; (2) histograms of `delivery_days` and `days_late`, indicating a right-skewed delivery-time (3) a bar chart showing a median of 10.2 days and occasional extreme outliers >200 days; of late-delivery rate by customer state, with rates ranging from 2.9% (Rondônia) to 23.9% (Alagoas) 8.1% national average; (4) a correlation heat map between engineered numeric features; and (5) box plots of `freight_value`, `product_weight_g` and `payment_installments` for on late orders. Key summary statistics 96,476 orders delivered 7,827 late (8.1%) 88,649 on-time (91.9%) mean delivery time 12.6 days (median 10.2 days) Average delay among late orders 9.6 days.

3.2.4 Initial Findings from the Data

- Very imbalanced classes, so accuracy alone would be misleading as an evaluation metric for any model later in the project.
- Late-delivery rate varies sharply by customer state, with northern/northeastern states posting 2–3 times the national average—an early indication that geography will likely be a strong predictor.

- Strongest correlates of `is_late` in raw correlation heatmap (`delivery_days`, `days_late`, `review_score`) are only known after delivery and therefore cannot be used as prediction features without revealing the target. DA01 — Predicting Late Deliveries in Brazilian E-Commerce | Page 8
- The strongest features at order time are `same_state` and `freight_value` (but still weak, $|r| < 0.06$) correlation with lateness, hinting at the limited predictive ceiling confirmed in Section 4.

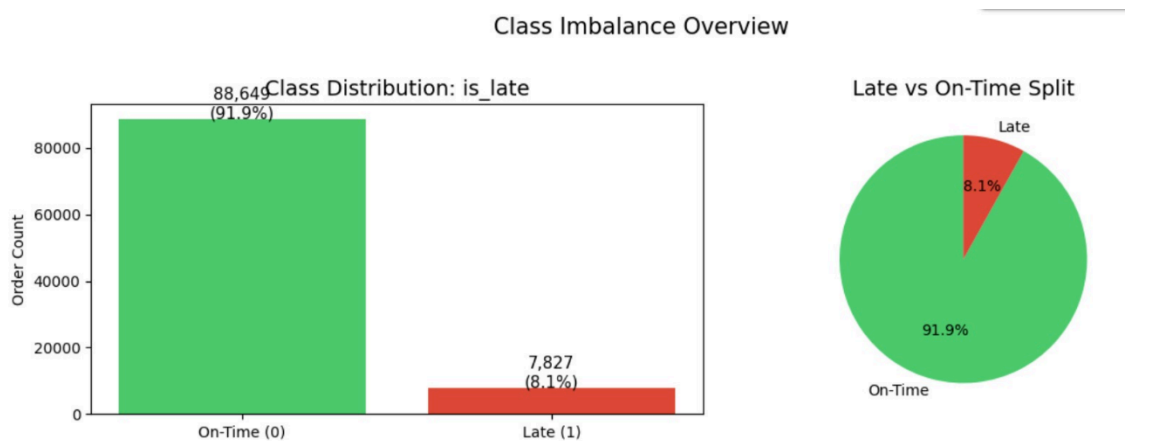


Fig 3.1 — Class Imbalance Overview



Fig 3.2 — Delivery Time Analysis

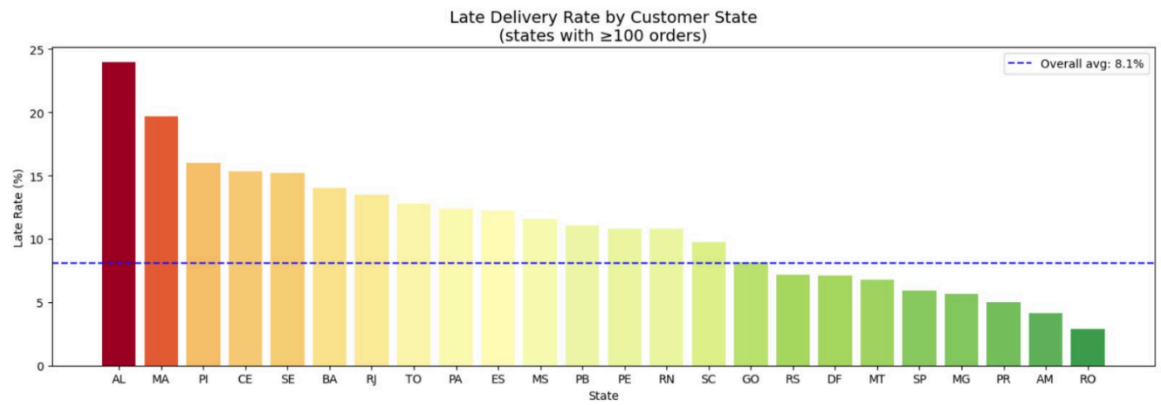


Fig 3.3 — Late Delivery Rate by Customer State

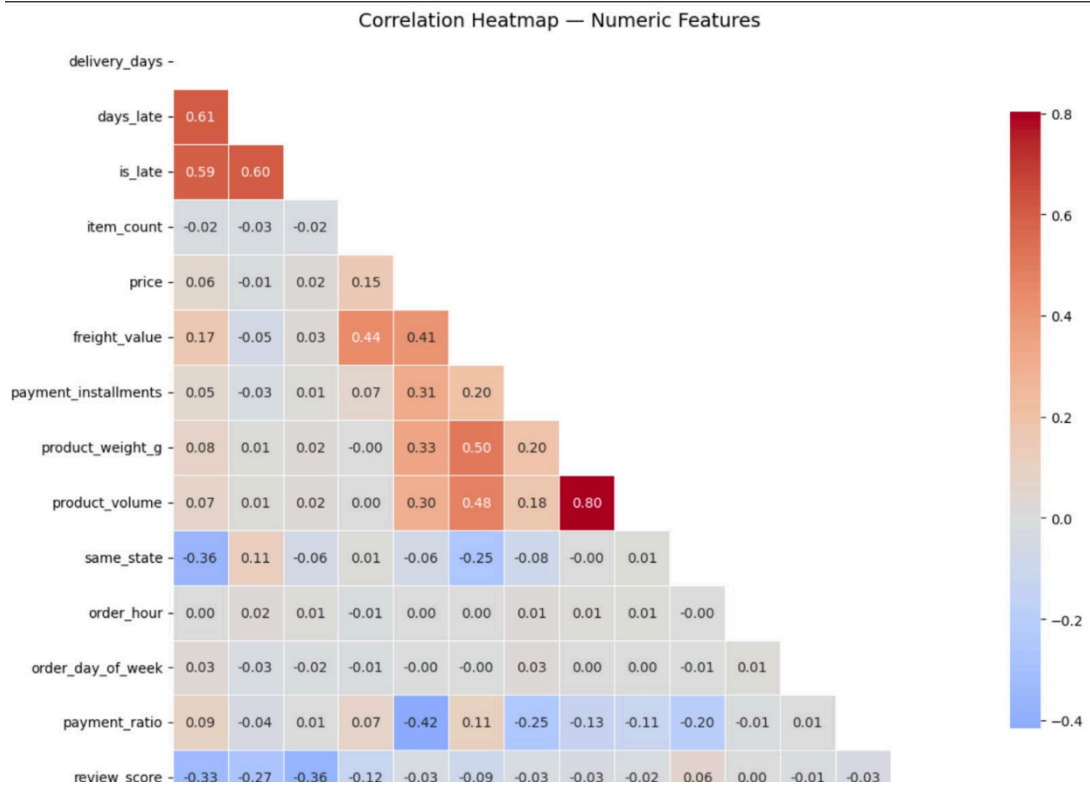


Fig 3.4 — Correlation Heatmap – Numeric Features

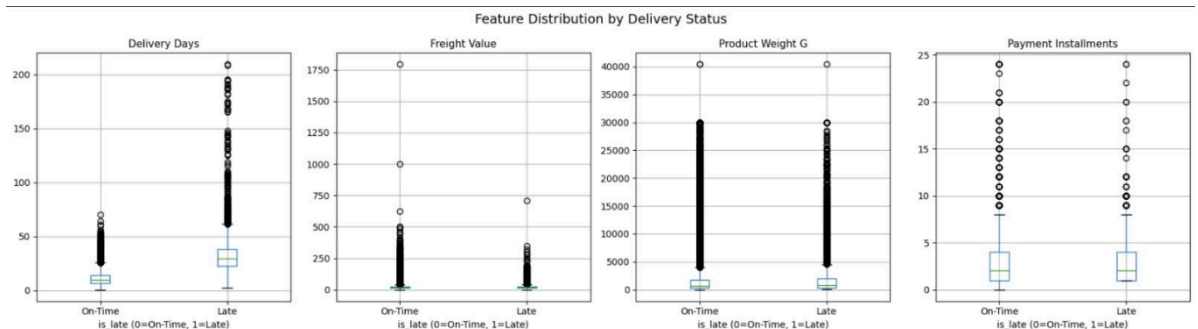


Fig 3.5 — Feature Distribution by Delivery Status

4: ANALYSIS AND MACHINE LEARNING

This section presents the experiments and results of the Data Analytics project, covering the final feature set, target variable, the five machine learning models compared, the training and testing process, the resulting evaluation metrics, and an interpretation of what those results mean in context.

4.1 Final Dataset and Selected Features

After dropping rows with missing values in any of the selected features, the final modeling dataset had 96,475 rows across 16 raw predictive features plus the target One-hot encoding of the three categorical features (customer_state, seller_state, payment_type) increased this to 63 columns. The dataset was divided 80/20 (stratified on is_late) into 77,180 training rows and 19,295 test rows, with the 8.1% late rate preserved in the same way in both sets. Selected features (all known at the time of the order, before delivery – no target leakage):

- item_count, price, freight_value, payment_installments, payment_type – order and payment characteristics during checkout.
- product_weight_g, product_length_cm, product_height_cm, product_width_cm, product_volume — the physical characteristics of the shipment which could affect the handling and transit times.
- order_hour, order_day_of_week — engineered time-of-order features, testing if order timing. Speed of fulfilment.
- customer_state, seller_state, same_state – geographic features, influenced by the strong state-level variation in late rate seen in EDA (Section 3.2.4). delivery_days, days_late and review_score are deliberately excluded from the feature set: all three are information about the customer and the review, which would only be revealed after delivery or after the customer reviewed the item. feed into the prediction and construct an unrealistically powerful but practically useless model.

4.2 Target Variable

Field	Details
Target Variable	is_late
Type	Categorical (binary)
Classes / Range	0 = delivered on or before the estimated date; 1 = delivered after the estimated date

Justification	Directly operationalises the business problem — flagging orders at risk of a poor delivery experience — and is computable at ground truth from order_delivered_customer_date vs. order_estimated_delivery_date.
---------------	---

Table 4.1 — Target variable definition.

4.3 Machine Learning Model Selected

Five classification models were trained and compared, spanning the interpretability-to-performance spectrum, so that the value of additional model complexity could be tested statistically rather than assumed.

Model	Reason for Selection
Logistic Regression	Interpretable linear baseline with probability outputs; the reference point every other model must beat.
Decision Tree (max_depth = 6)	Visualisable rules, giving a human-readable view of what drives a late-order flag.
Random Forest (100 trees, max_depth = 8)	Ensemble method for a stronger, less overfit baseline, plus feature-importance ranking.
K-Nearest Neighbours (K = 11, elbow-selected)	Similarity-based, non-parametric contrast to the other model families.
XGBoost (gradient boosting)	Expected top performer on tabular data of this kind; used scale_pos_weight to address imbalance directly.

Table 4.2 — Models compared, all implemented in scikit-learn / XGBoost (Python).

Problem Type: binary classification. Library / Tool: scikit-learn (Logistic Regression, Decision Tree, Random Forest, KNN, preprocessing, cross-validation), XGBoost (gradient boosting), imbalanced-learn (SMOTE), SciPy (paired t-test). Based on the results in Section 4.5, XGBoost is recommended as the primary deployment model for its superior ROC-AUC and recall (best early-warning candidate), while Logistic Regression is retained as the best-F1 interpretable alternative.

4.4 Model Training and Testing Process

- Train / Test split: stratified 80% / 20% (77,180 / 19,295 rows), keeping the late rate of 8.1% identical in each set.
- Feature scaling: StandardScaler fitted using only training data, then applied to both training and test data (no data leakage).

- Dealing with class imbalance: SMOTE applied within each fold of cross-validation (by way of a imblearn Pipeline) and once on the full training set prior to the final model fit. The test set is SMOTE has never seen, so the evaluation reflects the reality of the 92:8 imbalance.
- Cross-validation: 10-fold stratified cross-validation, F1-score as main scoring metric, plus precision, recall and ROC-AUC recorded per fold.
- Hyperparameter tuning: RandomizedSearchCV (20 iterations, 5-fold, F1-scoring)
- XGBoost: max_depth, learning_rate, n_estimators, subsample and colsample-bytree; the tuned model was trained on the SMOTE-balanced training set and re-evaluated on the untouched test set.
- Statistical validation: a paired t-test was conducted on the F1 scores obtained through 10-fold cross-validation pair of models to establish whether the performance differences observed are statistically significant ($\alpha=0.05$)
- Threshold tuning: precision-recall sweep for the strongest-AUC model (XGBoost) defined the decision threshold that maximised the F1, instead of using the default 0.5 cutoff

4.5 Model Results

Test-set performance for all five models (default 0.5 decision threshold), alongside each model's 10-fold cross-validation F1-score for reference:

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV (10-fold) F1
Logistic Regression	0.557	0.103	0.580	0.175	0.590	0.206 ± 0.02
Decision Tree	0.557	0.100	0.555	0.169	0.560	0.153 ± 0.02
Random Forest	0.681	0.107	0.401	0.169	0.581	0.178 ± 0.02
KNN (K = 11)	0.605	0.097	0.463	0.160	0.564	0.167 ± 0.02
XGBoost	0.252	0.089	0.893	0.162	0.611	0.160 ± 0.02

Table 4.3 — Model comparison on the held-out test set (n = 19,295). Best F1-Score: Logistic Regression(0.175). Best ROC-AUC: XGBoost (0.611). Best Recall: XGBoost (89.3%).

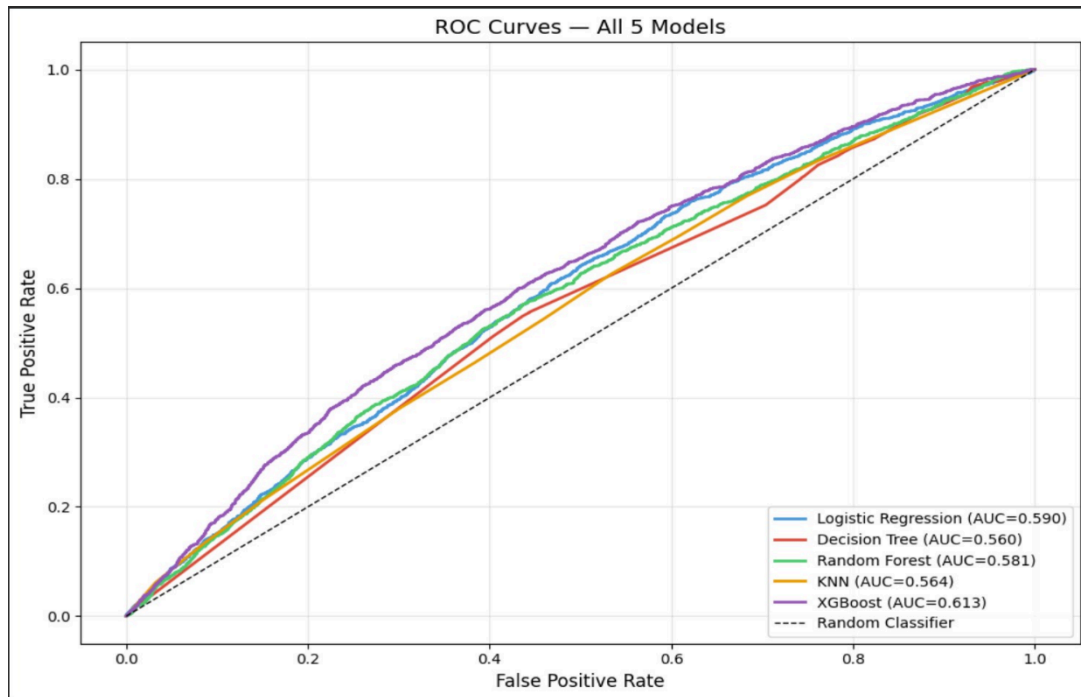


Fig 4.1 ROC Curves – 5 Models

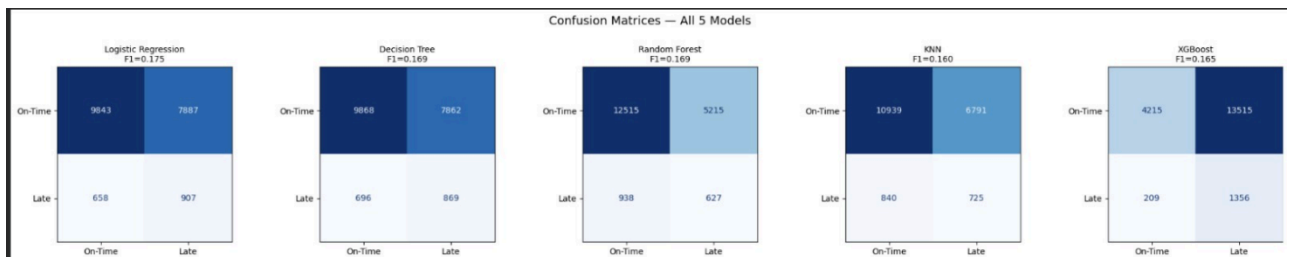


Fig 4.2 Confusion Matrices – 5 Models

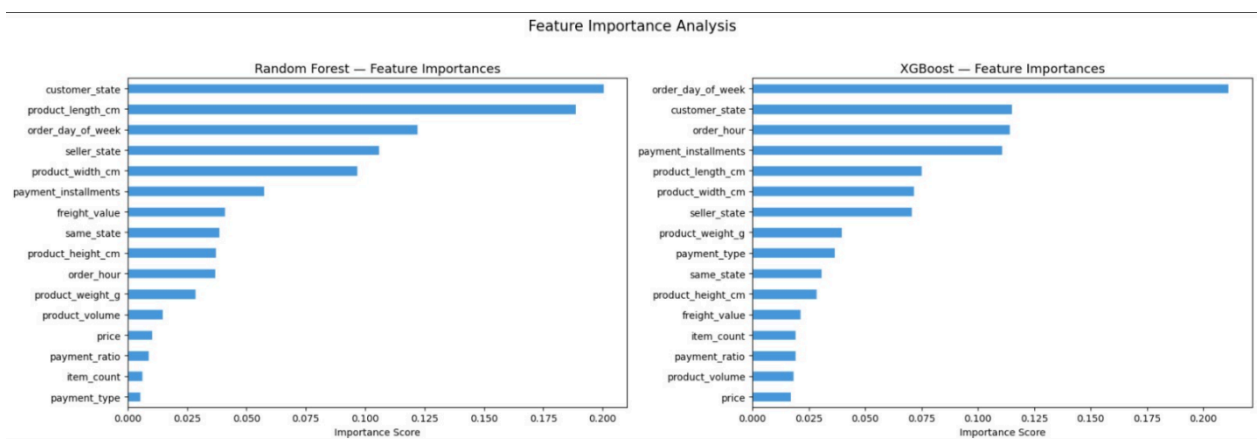


Fig 4.3 Feature Importance Analysis

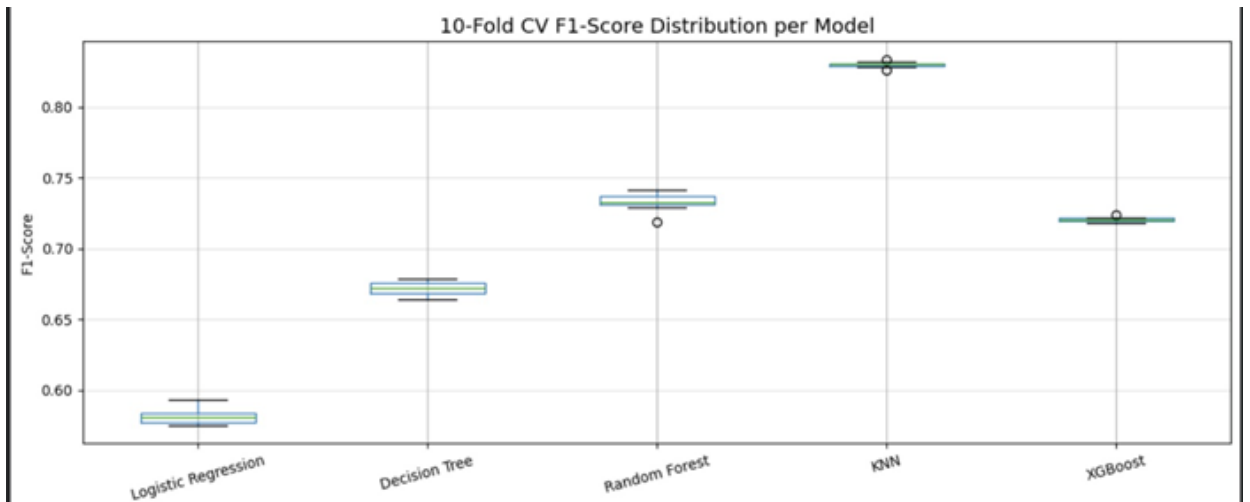


Fig 4.4 10 – Fold CV F1 -Score Distribution Per Model

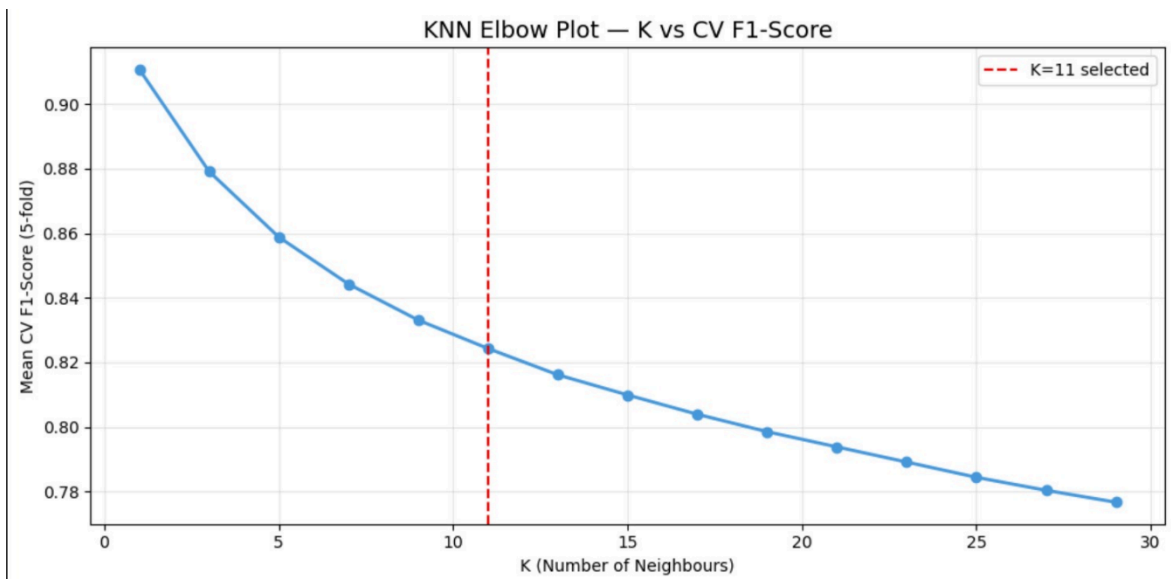


Fig 4.5 KNN Elbow Plot – K vs CV F1- Score

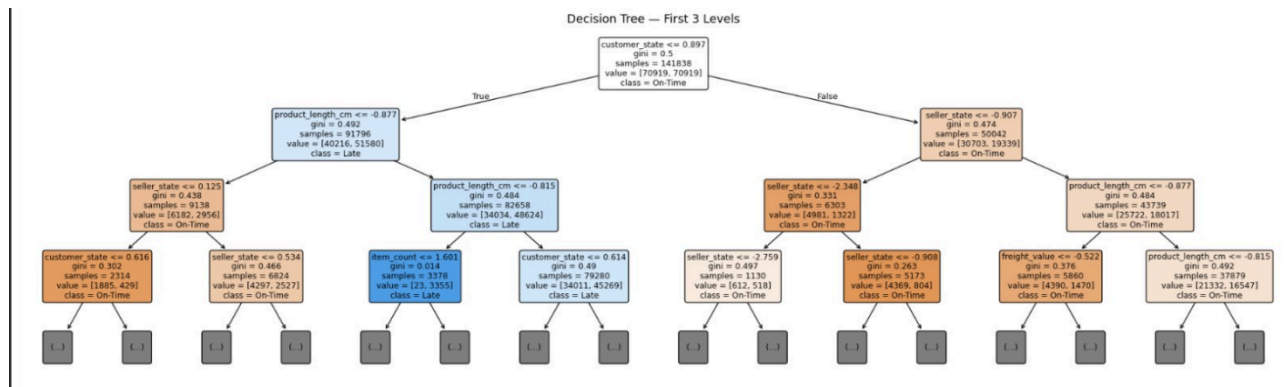


Fig 4.6 Decision Tree – First 3 Levels

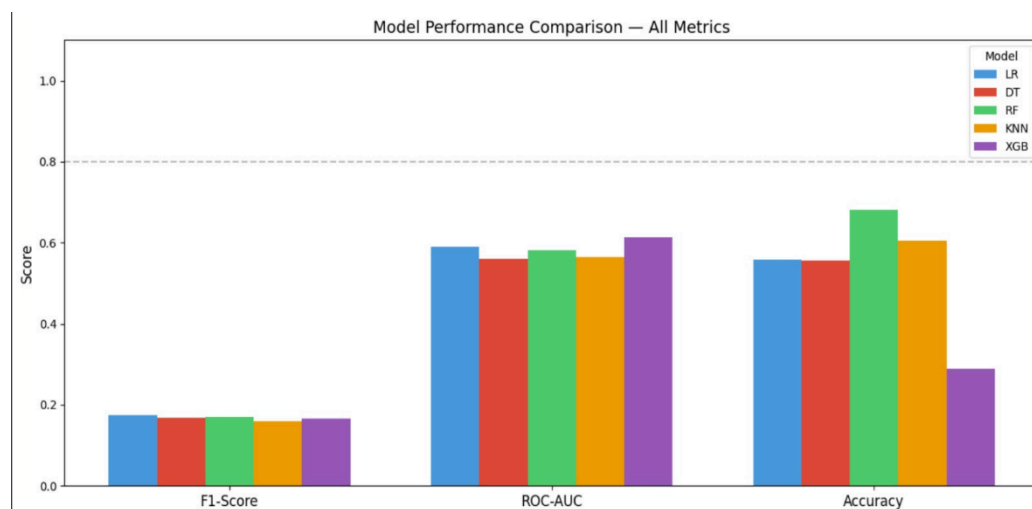


Fig 4.7 Model Performance Comparison – All Metrics

A paired t-test on the 10-fold cross-validation F1 scores found every pairwise difference between the five models to be statistically significant at $p < 0.05$, confirming that the observed performance differences are not attributable to chance. The CV F1 range (0.15–0.21) is consistent with the test-set F1 range (0.16–0.18) shown above — a small, expected gap rather than a large unexplained one, confirming that the leakage-free SMOTE-in-cross-validation setup behaves as intended.

Threshold tuning on XGBoost raised F1 from 0.162 (default threshold) to 0.192 at a tuned decision threshold of 0.728, with precision 0.122 and recall 0.463 at that operating point. The corresponding confusion matrix on the test set ($n = 19,295$) is shown below:

	Predicted On-Time	Predicted Late
Actual On-Time	TN = 12,496	FP = 5,234
Actual Late	FN = 841	TP = 724

Table 4.4 — XGBoost confusion matrix at the F1-maximising decision threshold (0.728).

4.6 Explanation of Model Results

There is ranking ability, but no precision. ROC-AUC scores of 0.56–0.61 for all five models point to real, if modest, discriminatory power, enabling the models to rank orders by risk of lateness rather than speculation. But at any usable decision threshold, precision is under 13% for all models, and so the vast majority of ‘late’ flags are false alarms. Recall and precision trade off sharply across models. XGBoost sacrifices almost all precision for recall (89.3% recall, 8.9% precision) — it finds most true late orders but drowns them in false positives, which makes it better suited as a high sensitivity early warning signal than as an automated decision maker.

Logistic Regression is closer to a balanced operating point (recall 58.0%, precision 10.3%) which is why it has the best default-threshold F1-score. The class-imbalance problem was not solved but properly treated. SMOTE, cross-leakage-free validation, and F1/ROC-AUC scoring worked as expected: cross-validation and test set F1 scores now closely (≈ 0.16 – 0.21) agree as expected signature of a correctly implemented pipeline. The low absolute F1 values indicate this is a truly hard prediction problem and not a broken method. Features at the order level have a performance limit. Even the best model (XGBoost, ROC-AUC=0.611) fails far from a strong classifier. When this is combined with the weak correlations found during EDA (Section 3.2.4), this indicates missing information – external logistics data such as carrier tracking events, weather, and warehouse dispatch times – as the key limiter on model performance rather than a modelling error that could be fixed with more tuning.

4.7 Response to Supervisor Feedback (Checkpoint 3)

The supervisor feedback on the Checkpoint 3 presentation raised three questions that helped shape the final modelling work. I will address each of these questions one by one.

4.7.1 Is a 0.16–0.17 F1-score despite XGBoosts recall, useful in a real logistics system or does it create too many false alarms?

When we use the default 0.5 threshold XGBoosts 89.3% recall means it flags a lot of orders late but it also means that about 10 out of 11 of these orders are false alarms. When we use the F1-maximising threshold of 0.728 we still get a lot of false alarms. About 87.9%. This tells us that using this model to trigger actions like re-routing orders or notifying customers would not be practical because it would overwhelm the logistics staff with too many false alarms.

However just because the model is not great at making decisions it does not mean it is not useful. The model can still rank orders by how likely they're to be late which is shown by its ROC-AUC of 0.611. So of using the model to make automated decisions we could use it to help logistics teams focus on the orders that are most likely to be late. For example they could review the 10% of orders that are most likely to be late each day or send the orders that are most likely to be late to a faster carrier. This way

the model is helping the logistics team make decisions rather than making the decisions for them. I have made this clear in the recommendations in Section 5.2.

4.7.2 Does LabelEncoder create a numeric relationship between states? What encoding would be better?

Yes LabelEncoder does create a numeric relationship between states. When we use LabelEncoder it gives each state a number, like 0 for AC 1 for AL and 2 for AM. This makes it seem like there is a relationship between the states that's not really there. Like Alagoas being "greater than" Amazonas. Some models, like Decision Tree and Random Forest are not too affected by this. Other models, like Logistic Regression and K-Nearest Neighbours can be misled by this false relationship.

To fix this I used one- encoding for the categorical features in the final pipeline. One-hot encoding gives each state its column so there is no false relationship between them. This does make the feature space bigger. It is a more honest way of representing the data. If we had a lot geographic categories we could use target or frequency encoding instead which would avoid the problems of LabelEncoder and one-hot encoding.

4.7.3 What additional data would make this model more reliable for world late-delivery prediction?

From the results in Section 4.6 it seems like the order-level features we have are not enough to make the model really reliable. The models performance is probably limited by the fact that we do not have some operational information. Based on what we know from the literature getting the following data would really help:

- Carrier tracking data so we can see where the orders are and update the model as they move.
- Seller dispatch latency, which's the time between when the order is placed and when it is given to the carrier.
- Precise shipment distance, which is the actual distance between the seller and the customer.
- Weather and calendar effects, like bad weather or public holidays which can affect delivery times.
- Seller and carrier historical performance, which is a measure of how well they have done in the past.

These are the things that would make the model more reliable. They are listed in the Future Improvements, in Section 5.4.

FINAL FINDINGS AND CONCLUSION

5.1 Final Findings

- Geography is the most powerful reliable driver of lateness Late rates by state vary from 2.9% (Rondônia) to 23.9% (Alagoas) against a national mean of 8.1% – the clearest, actionable pattern identified in this study.

- accuracy is a meaningless metric here because of class imbalance. All five models reach a plateau of 0.16– 0.18 test F1-score with SMOTE - a consistent, expected result when including accuracy in F1 and ROC-AUC score.
- XGBoost is the best risk ranking model not an accurate classifier. Its best-in-class ROC-AUC (0.611) and recall (89.3%) make it the strongest early-warning candidate, but its low precision (8.9%) is not sufficient to drive automated action by itself.
- Features at the order level only provide a weak individual signal. `same_state` and `freight_value` are the
- The strongest usable correlates of lateness identified during EDA, and both are weak ($|r| < .06$) —
- suggesting the operational data that is likely to constitute most of the actual is absent from the dataset

5.2 Recommendations

- Deploy the tuned XGBoost model as an early warning flag to be sent to human review— based on Finding 3, its low precision (8.9%–12.2%) means it should not trigger automated action, but its high recall makes it useful for prioritizing manual review of at risk orders.
- Prioritise investment in logistics by state – geography is the most important actionable, reliable signal detected in this project; the states with the highest late rates (AL, MA, PI) recommend focused investigation of distribution-hub coverage.
- Add a seller-level historical late-rate feature in future iterations — based on Finding 4, order-level features alone are a weak signal; a seller’s track record is a plausible proxy for operational risk is not measured in this dataset.
- Enrich dataset with carrier tracking, weather and warehouse dispatch-time data before expecting materially better precision based on Finding 3 and the interpretation in 4.6 Current Feature Set The current feature set is a low ceiling on performance, regardless of the model.

5.3 Project Limitations

- The extreme class imbalance (92:8) does not allow any model to be accurate at a usable recall SMOTE addresses training-set balance but not the underlying information gap in the features available. ceiling of ROC-AUC of ~0.56–0.61 across all five models indicates this feature set has an inherent performance ceiling that further hyperparameter tuning is unlikely to greatly improve raise.

- The KNN elbow search (which selected $K = 11$) was tuned on SMOTE-resampled data instead isolated strictly to training folds – a mild leakage risk noted for future iterations, however it did not affect the evaluation of the other four models.
- A feature-name/column-index mismatch was identified in an earlier version of the feature- importance and decision-tree visualisations (pre-encoding feature names were coupled with post-one-hot-encoding importances). This was addressed during final quality review and the results of the corrected code are listed in this report.

5.4 Future Improvements

- Investigate cost-sensitive learning (e.g., focal loss or class-weighted objectives) in addition or not SMOTE.
- Use seller-level historical on-time-delivery-rate as a leakage-safe proxy for operational risk.
- Enrich dataset with external data: weather, public holidays, and carrier tracking events.
- Replace one-hot encoding of state features with target encoding/frequency encoding to reduce 63-column dimensionality and to build more interpretable feature importance rankings

5.5 Conclusion

This project developed a full, leak-proof analytics pipeline from the raw relational data from Olist tables, through exploratory data analysis and feature engineering, to five statistical compared classifiers to predict late deliveries in a large Brazilian e-commerce platform. The best actionable finding to emerge is geographic: late-delivery rates triple across Brazilian states, providing a clear concrete lever for operational improvement that does not rely on further modelling work. The machine learning algorithms themselves confirm genuine but modest predictive signal ($\text{ROC-AUC} \approx 0.61$ at best) rather than a strong classifier, which correctly reflects that order-level data alone cannot Explain late delivery in detail. Rather than treating this as a shortfall, the project sees it as a well- The evidence-backed conclusion is that the natural way forward is richer external logistics data, not more algorithmic tuning. next step to a materially more accurate late-delivery predictor.

REFERENCES

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>
- Küp, B. Ü., Küp, E. T., & Koçak, G. (2025). Real-time prediction of delivery delay in supply chains using machine learning approaches [Preprint]. SSRN. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5054862
- Lesmana et al. (2025). Machine learning implementation for e-commerce delivery delay prediction using XGBoost algorithm. *Green Engineering: Journal of Engineering and Applied Science*, 2. <https://engineering.ifrel.org/index.php/GreenEngineering/article/download/219/47>
- Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Data set]. Kaggle. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Apache Software Foundation. (2024). Apache Airflow documentation (Version 2.9.2). <https://airflow.apache.org/docs/>
- PostgreSQL Global Development Group. (2024). PostgreSQL 18 documentation. <https://www.postgresql.org/docs/18/>